

Pong en Red

Documentacion del codigo y del protocolo
Proyecto N1 - Internet: Arquitectura y Protocolos

Arquitectura cliente/servidor - Sockets Berkeley (TCP)
Servidor en C - Cliente en Python (Tkinter)

1. Introduccion

Este proyecto implementa el clasico juego **Pong** en red, con una arquitectura **cliente/servidor**. El servidor mantiene el estado del juego y comunica a los clientes mediante un protocolo binario propio, **MyAppGameProtocol**, que corre sobre TCP usando la API de Sockets Berkeley. El servidor esta escrito en **C** (requisito del enunciado) y el cliente en **Python** con Tkinter, por lo que corre igual en Windows y Mac.

Decisiones principales

TCP (SOCK_STREAM): se elige porque los comandos del juego (registro, movimientos, marcador) no se pueden perder ni desordenar; TCP garantiza entrega confiable y en orden.

Protocolo binario: los mensajes viajan como bytes crudos (no texto), tal como pide el enunciado.

Arquitectura desacoplada: cada responsabilidad (red, protocolo, logica de juego, interfaz) vive en su propio archivo.

2. Arquitectura desacoplada (Diagrama de componentes)

El protocolo no sabe nada del juego ni de los sockets: solo convierte mensajes en bytes y viceversa. Esto permite cambiar una capa sin tocar las otras. El siguiente diagrama muestra los modulos y como se comunican.

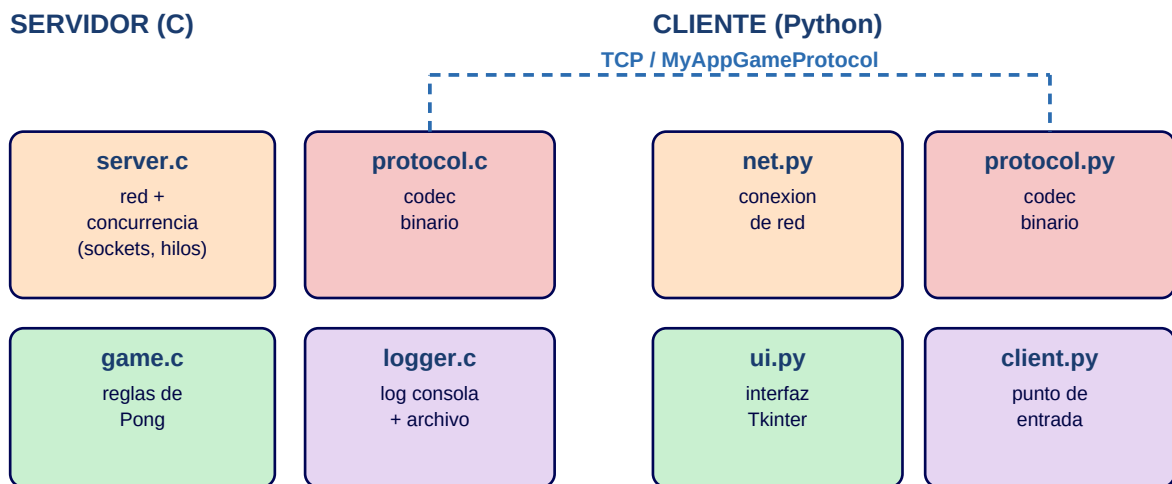


Figura 1. Componentes del servidor (C) y del cliente (Python).

3. MyAppGameProtocol (protocolo propio y binario)

3.1 Formato del mensaje

Todo mensaje tiene una cabecera fija de 3 bytes seguida de un payload de tamaño variable:



Cabecera fija de 3 bytes + payload de tamaño variable

Figura 2. Formato binario del mensaje.

type (1 byte): tipo de mensaje. **length** (2 bytes, big-endian): tamaño del payload. **payload**: los datos, cuyo contenido depende del tipo.

3.2 Vocabulario de mensajes

Tipo	Nombre	Dirección	Para que sirve
1	REGISTER	Cliente -> Servidor	Enviar nickname y email
2	WELCOME	Servidor -> Cliente	Asignar número de jugador (1 o 2)
3	WAITING	Servidor -> Cliente	Esperando al otro jugador
4	START	Servidor -> Cliente	La partida empieza
5	INPUT	Cliente -> Servidor	Mover paleta (arriba/abajo/quieto)
6	STATE	Servidor -> Cliente	Estado: pelota, paletas y marcador
7	GAMEOVER	Servidor -> Cliente	Fin de partida y ganador
8	ERROR	Servidor -> Cliente	Reportar un error

Tabla 1. Vocabulario de mensajes de MyAppGameProtocol.

3.3 Payload de STATE (10 bytes)

ball_x (2) + ball_y (2) + paddle1_y (2) + paddle2_y (2) + score1 (1) + score2 (1). Las coordenadas usan un tablero lógico de 640 x 480.

4. Reglas de procedimiento (Diagrama de secuencia)

El siguiente diagrama muestra el intercambio de mensajes desde que los dos jugadores se conectan hasta que termina la partida.

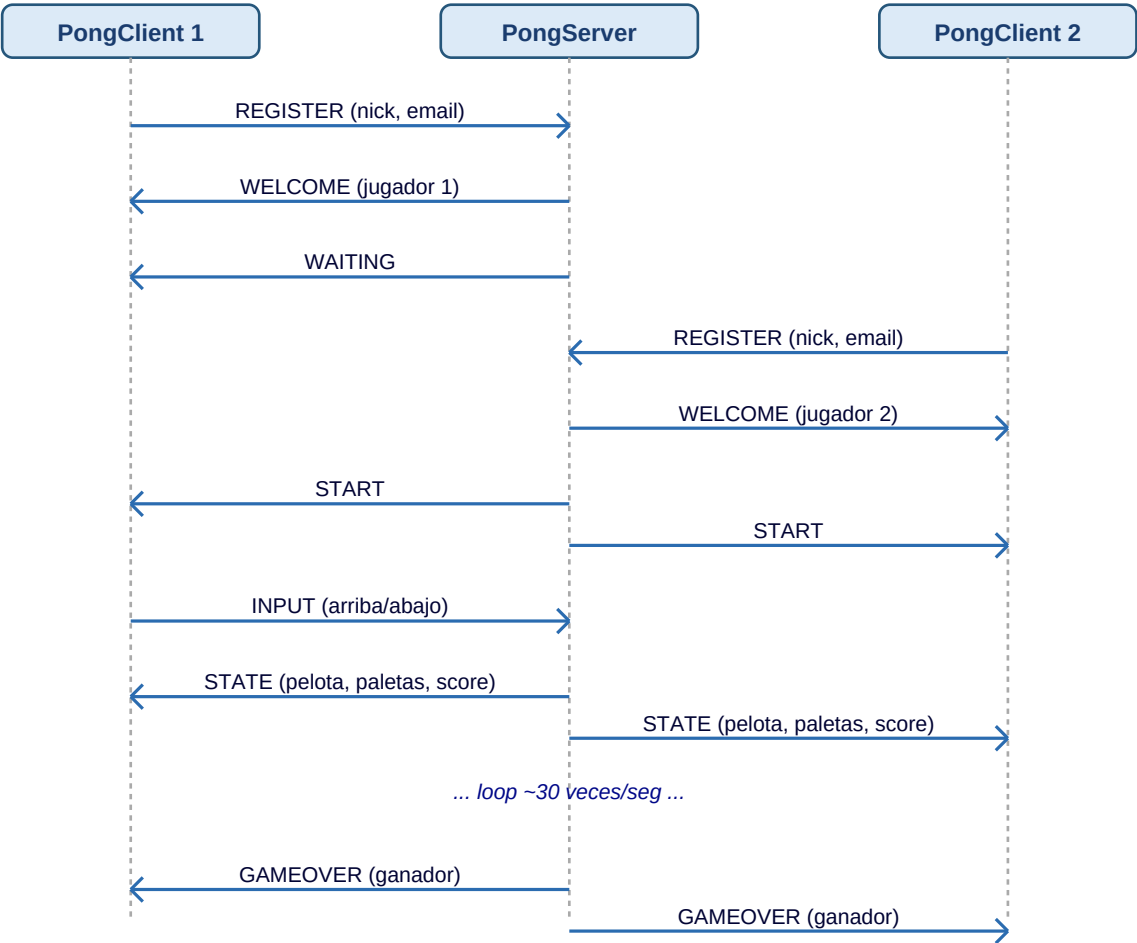


Figura 3. Secuencia de mensajes entre clientes y servidor.

5. Explicacion del codigo por seccion

5.1 protocol.c / protocol.py - Codec binario

Convierte mensajes en bytes y viceversa. En C, **proto_send** arma la cabecera byte a byte y **proto_recv** la reconstruye. En Python se usa **struct.pack('>BH', ...)** (B=1 byte, H=2 bytes, >=big-endian).

```
>>> C (armar la cabecera)
header[0] = type; // tipo
header[1] = (length >> 8) & 0xFF; // byte alto
header[2] = length & 0xFF; // byte bajo

>>> Python (empaquetar en binario)
struct.pack('>BH', msg_type, len(payload)) + payload
```

5.2 game.c - Logica del juego

Contiene solo las reglas de Pong (mover paletas, rebotes, marcador). No sabe nada de red ni de bytes, por eso es facil de probar y explicar.

```
g->ball_x += g->ball_vx; // mover la pelota
g->ball_y += g->ball_vy;
if (g->ball_y <= 0 || g->ball_y >= BOARD_H)
g->ball_vy = -g->ball_vy; // rebote arriba/abajo
```

5.3 server.c - Red y concurrencia

Crea el socket TCP (socket/bind/listen/accept), atiende a cada cliente en un hilo, empareja jugadores y corre el game loop de cada partida. Soporta varias parejas jugando en paralelo gracias a los hilos (pthreads).

```
./server <PORT> <LogFile> # como se ejecuta
server_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(...); listen(...); accept(...);
pthread_create(&t, NULL, client_thread, ca); // un hilo por cliente
```

5.4 logger.c - Registro de eventos

Imprime cada evento en consola y lo guarda en el archivo de log con fecha y hora. Usa un mutex para que los hilos no mezclen sus lineas.

5.5 Cliente Python (net.py, ui.py, client.py)

net.py maneja la conexion (enviar/recibir por el socket). **ui.py** dibuja el juego con Tkinter y captura las teclas (flechas o W/S). **client.py** pide nickname/email, se registra y abre la ventana.

6. Como ejecutar

Servidor (compilar y correr):

```
cd server
make
./server 5000 pong.log
```

Cliente (en cada maquina Windows/Mac con Python 3):

```
cd client
python client.py <IP_DEL_SERVIDOR> 5000
```

Se abren dos clientes para jugar una partida. Controles: flechas o teclas W / S. El despliegue en AWS esta detallado en docs/AWS_DEPLOY.md.

7. Conclusiones

TCP encaja con Pong porque garantiza entrega y orden de los comandos. Separar red, protocolo, logica y logging hace el codigo corto, mantenible y facil de explicar. Un protocolo binario con cabecera [tipo][longitud] es simple de codificar en C y en Python, y los hilos permiten atender varias parejas de jugadores de forma concurrente.

8. Referencias

Beej's Guide to Network Programming - <https://beej.us/guide/bgnet/>

TCP Server-Client en C - [geeksforgeeks.org/tcp-server-client-implementation-in-c](https://www.geeksforgeeks.org/tcp-server-client-implementation-in-c/)

Pong (Wikipedia) - <https://en.wikipedia.org/wiki/Pong>